VCube Software Solutions Pvt Ltd

Object Oriented Programming

Dr Raju. Nadimpalli.

Introduction to Object Oriented Programmin

- Most traditional development methodologies are either **algorithm centric or datacentric**.
- In an **algorithmic-centric methodology**, we think of an algorithm that can accomplish the task, then build data structures for that algorithm to use.
- In a **data-centric methodology**, we think how to structure the data, then build the algorithm around that structure.
- In an **Object-Oriented methodology** the algorithm and data-structures are packed together as an object, which has a set of attributes and properties.

Object and Class

- Everything in the universe is an object. (Any real-world entity; either tangible / intangible)
- Every object has three characteristics: **Identity, State and Behavior**.

Identity : Name of the object

State : Properties / Attributes

Behavior : actions / Functions / Methods/

Responsibilities

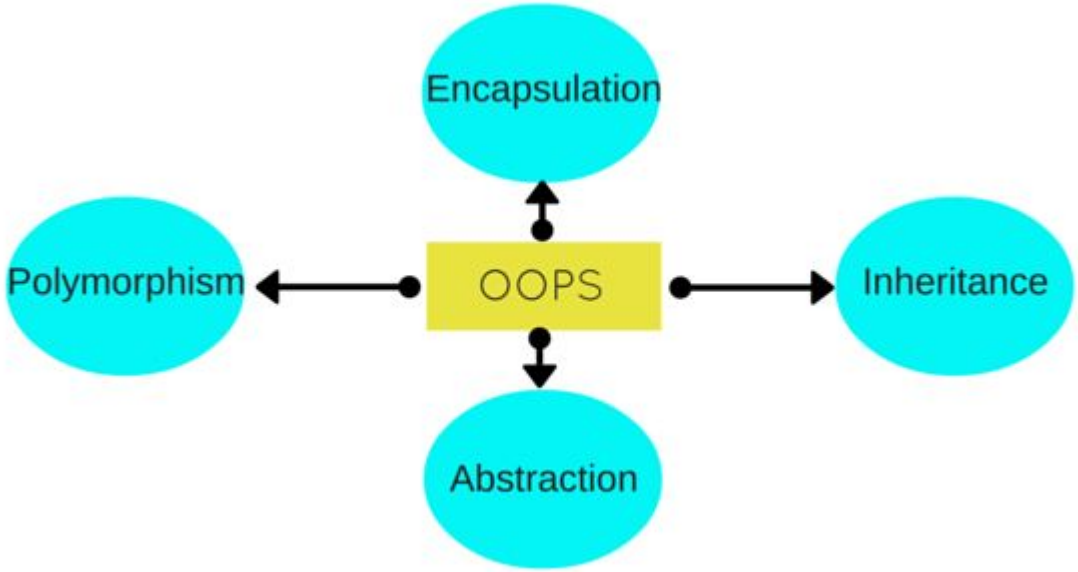
```
class class_name:
    def ...

    ..
```

- In general object is called as combination of data members and members functions
- A class is a user defined datatype, is a collection of objects; all objects share same properties and behaviors.
- In programming terms: Instance of a class is an object.

Principles of Object Oriented Programm

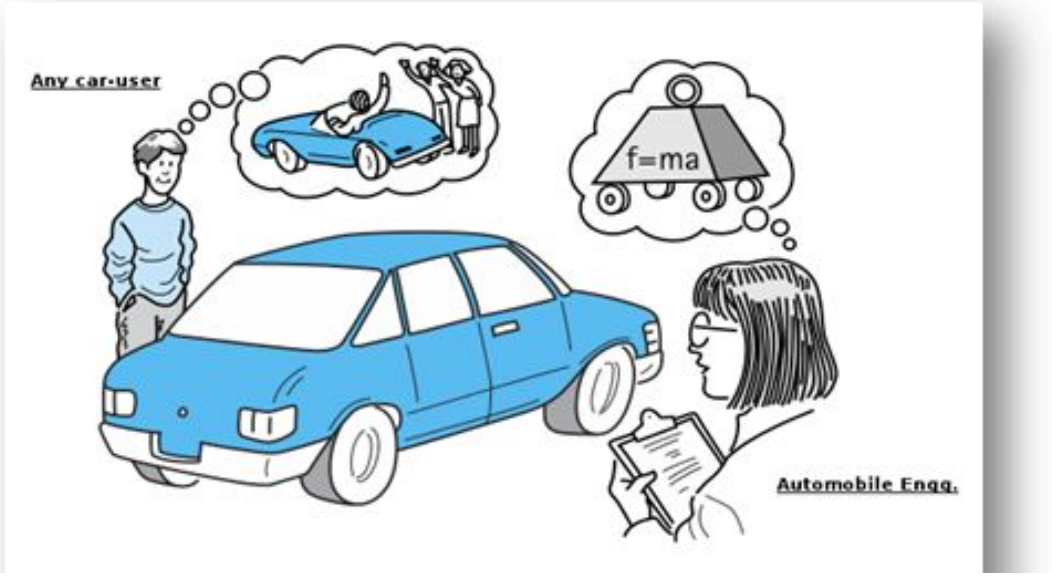
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

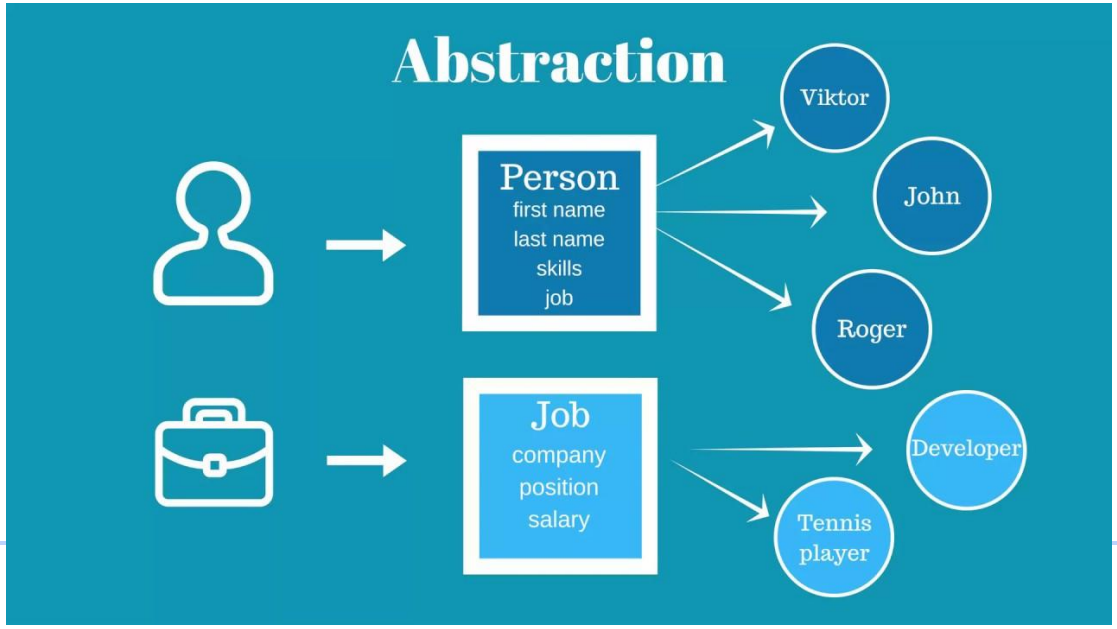


Abstraction

- Data Abstraction is a process of providing only the essential details to the outside world and hiding the internal details, i.e., representing only the essential details in the program.
- Data Abstraction is a programming technique that depends on the separation of the interface and implementation details of the program.
- Class definition is best example for abstraction in Java/C++/Python.

Abstraction

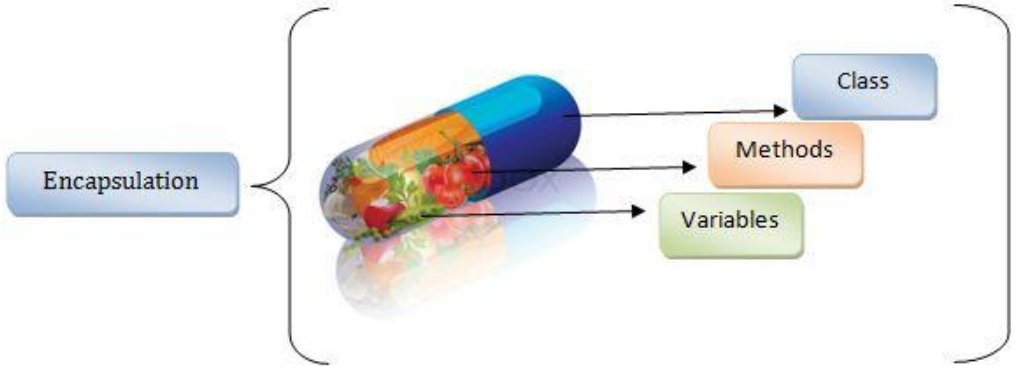


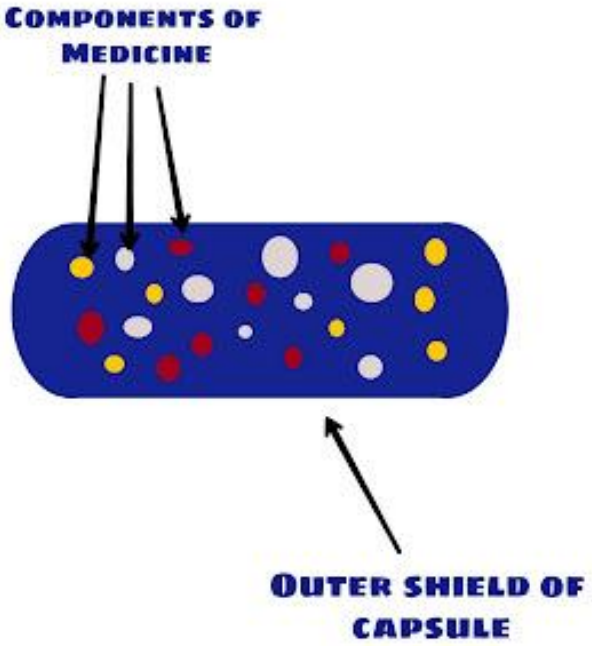


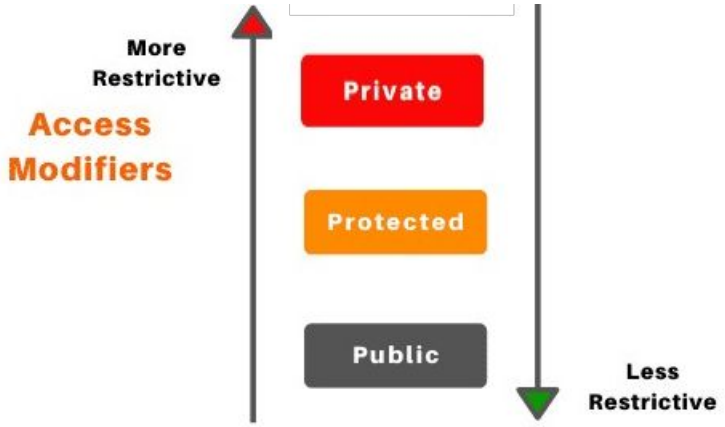
Encapsulation

- Encapsulation is the mechanism that binds together code and data it manipulates and keeps both safe from outside interference and misuse.
- We can create a fully encapsulated class by making all the data members of the class private.
- Now we can use setter and getter methods to set and get the data in it.
- By providing only setter or getter method, you can make the class read-only or write-only.

Encapsulation







Inheritance

- **Inheritance** is a mechanism in which one object acquires all the properties and behaviors of a parent object.
- The idea behind inheritance is that you can create new classes that are built upon existing classes.
- When you inherit from an existing class, you can reuse methods and fields of the parent class.
- Moreover, you can add new methods and fields in your current class also.

Polymorphism

- **Polymorphism,** we can perform a *single action in different ways*.
- Polymorphism : "poly" means many and "morphs" means forms. So polymorphism means many forms.
- There are two types of polymorphism : compile-time polymorphism and runtime polymorphism.
- We can perform polymorphism by method overloading and method overriding.
- Method overloading is an example of compile time and method overriding is an example of runtime polymorphism.

Python is objected-oriented

•Python is an object-oriented programming language; key notion is that of an object.

•An object consists of two things: ***data*** and ***functions*** (called methods) that work with that data.

- **strings** in Python are objects.
- •The data of the string object is the actual characters that make up that string.
- •The methods are things like lower(), replace(), split()…..

• In Python, everything is an object.

- That includes not only strings and lists, but also integers, floats, and even functions themselves.

Creating own classes: Steps

- A class is a template for objects. It contains the code for all the object's methods.
- To define a class in Python, you can use the ***class*** keyword, followed by the ***class_name*** and a ***colon***.
- Inside the class, an **__init__** method has to be defined with ***def***.
- The underscores indicate that it is a special kind of method.
- It is called a constructor, and it is automatically called when someone creates a new object from your class.
- The constructor is usually used to set up the class's variables
- **__init__** must always be present. It takes one argument: ***self***, which refers to the object itself.
- To instantiate an object, type the class name, followed by two brackets. We can assign this to a variable.

Note: **self** in Python is equivalent to **this** in C++ or Java.

Creating objects

- Object refers to a particular instance of a class where the object contains variables and methods defined in the class.
- Class objects support two kinds of operations: attribute references and instantiation.
- The term attribute refers to any name (variables or methods) following a dot.
- The act of creating an object from a class is called *instantiation*.

Attribute References

- The names in a class are referenced by objects called ***attribute references.***
- *There are* two kinds of ***attribute references:***

  data attributes and ***method attributes***.
- Variables defined within the methods are called ***instance variables*** *and are used to store data values.*
- *New* instance variables are associated with each of the objects that are created for a class. These instance variables are also called ***data attributes.***
- **Method attributes** *are methods inside a* class and are referenced by ***objects of a class.***
- **Attribute references use the standard dot notation syntax as supported in Python.**

Accessing attributes

- The syntax to access data attribute is,

  object_name.data_attribute_name

- The syntax to assign value to data attribute is,

  object_name.data_attribute_name = value

  where value can be of integer, float, string types, or another object itself.

- The syntax to call method attribute is,

  object_name.method_attribute_name()

Class instantiation

object_name = ClassName(argument_1, argument_2, ..., argument_n)

- The syntax creates a new object for the class ClassName and assigns object to the variable object_name.
- We can specify any number of arguments during instantiation of the class object.
- Note:
- Method definitions have ***self*** as the first parameter.
- This is because **whenever we call a method using an object, the object itself is automatically passed in as the first parameter to the self parameter variable**.

Example for a class and object

```
class Person:

    def __init__(self):
        print("In Constructor method")

    def receive(self):
        print("Receive method")

    def send(self):
        print("Send method")

p = Person()

p.receive()
p.send()
```

- The first parameter in each method must be ***self***.

- *When self is used, it is just a variable name to which the* object that was created based on a class is assigned.

In Constructor method

Receive method

Send method

Passing arguments to functions

```
class Addition:

    def __init__(self, a, b):
        self.a = a
        self.b = b

    def add(self):
        return self.a + self.b

a = Addition (10, 20)
print(a.add())
```

- To create a class, we use the **class statement.**
- Most classes will have a method called **__init__.**
- The first argument to every method in your class is a special variable called **self.**
- Every time your class refers to one of its variables or methods, it must precede them by self.
- To create a new object from the class, you call the class name along with any values that you want to send to the constructor.
- To use the object's methods, use the dot operator,

Constructor methods

•Python uses a special method called a constructor method. Python allows you to define only one constructor per class.

•Also known as the __init__() method, it will be the first method definition of a class.

```
def __init__(self, parameter_1, parameter_2, ...., parameter_n):
    statements
```

•The __init__() method defines and initializes the instance variables.

•It is invoked as soon as an object of a class is instantiated

Example for constructor

```
class Person:
    def __init__(self, name):
        self.person_name = name

    def receive(self):
        print(f"Receive method {self.person_name} Person")

    def send(self):
        print(f"Send method {self.person_name} Person")

p = Person("Ram")
p.receive()
p.send()
```

```
Receive method Ram Person
Send method Ram Person
```

Working with multiple objects

- Multiple objects for a class can be created while attaching a unique copy of data attributes and methods of the class to each of these objects.

```
class Person:
    def __init__(self, name):
        self.person_name = name

    def receive(self):
        print(f"Receive method {self.person_name} Person")

    def send(self):
        print(f"Send method {self.person_name} Person")

p = Person("Ram")
p.receive()
p.send()

p2 = Person("Krishna")
p2.receive()
p2.send()
```

```
Receive method Ram Person
Send method Ram Person

Receive method Krishna Person
Send method Krishna Person
```

Deleting objects

- We can delete the properties of the object or object itself by using the del keyword.

```
class Person:
    def __init__(self, name):
        self.person_name = name

    def receive(self):
        print(f"Receive method {self.person_name} Person")

    def send(self):
        print(f"Send method {self.person_name} Person")

p = Person("Ram")
p.receive()
p.send()

del p
p.receive()
p.send()
```

```
Receive method Ram Person
Send method Ram Person

---------------------------------------------------------------
NameError                                 Traceback
<ipython-input-9-7dcb3f4869cf> in <module>
     14
     15 del p
---> 16 p.receive()
     17 p.send()

NameError: name 'p' is not defined
```

Object assignment

- We can assign one object to another using the assignment **operator in python**.

```
class Person:

    def __init__(self):
        print("In Constructor method")

    def receive(self):
        print("Receive method")

    def send(self):
        print("Send method")

p = Person()
p.receive()

p1 = p
p1.send()
```

```
class Person:

    def __init__(self):
        print("In Constructor method")

    def receive(self):
        print("Receive method")

    def send(self):
        print("Send method")

p = Person()

p.receive()

p1 = p

p1.send()
```

```
In Constructor method
Receive method
Send method
```

Checking IDs of objects

```
class Person:
    def __init__(self):
        print("In Constructor method")
    def receive(self):
        print("Receive method")
    def send(self):
        print("Send method")

p = Person()
p.receive()

p1 = p
p1.send()

print(id(p))
print(id(p1))
print(id(p)==id(p1))
```

```
In Constructor method
Receive method
Send method
654462957320
654462957320
True
```

Objects as arguments

- An object can be passed to a calling function as an argument.
- When we pass an object of a class as an argument, we have access to all the data attributes attached to that object.

Objects as arguments and returned v

```
class Student:
    def __init__(self, name):
        self.name = name

class Person:
    def __init__(self, student):
        self.student = student

    def showStudent(self):
        print("Student Name: ",self.student.name)

    def getStudent(self):
        return self.student

s1 = Student("Ram")
p1 = Person(s1)
p1.showStudent()

old_s1 = p1.getStudent()
print("Student Name : ",old_s1.name)
```

```
Student Name:  Ram
Student Name :  Ram
```

Object-Oriented Programming

Part 2

Dr Raju

Demonstrate Class members and Instance members

```
class student:
    college = “Naresh Tech"  # class variable shared by all instances

    def __init__(self, name):
      self.name= name         # instance variable unique to each
instance

stud1 = student('Rama')

stud2 = student('Krishna')

print(student.college)

print(stud1.college)          # shared by all students

print(stud2.college)          # shared by all students

print(stud1.name)             # unique to stud1

print(stud2.name)             # unique to stud1
```

Demonstrate Instance members

```
class student:

    def __init__(self):
        self.sub1=0
        self.sub2=0

    def Compute_Avg(self):
        res = (self.sub1 + self.sub2)/2;
        print(f"Result is {res}")

stud1 = student()
stud1.Compute_Avg()

stud2 = student()
stud2.sub1=88
stud2.sub2=90

stud2.Compute_Avg()
```

```
Result is 0.0
Result is 89.0
```

Abstraction and Encapsulation

- Abstraction is a process where you show only “relevant” variables that are used to access data and “hide” implementation details of an object from the user.
- Encapsulation is the process of combining variables that store data and methods that work on those variables into a single unit called class.
- In Encapsulation, the variables are not accessed directly; it is accessed through the methods present in the class.
- Encapsulation ensures that the object’s internal representation (its state and behavior) are hidden from the rest of the application.
- Thus, encapsulation makes the concept of data hiding possible.

Abstraction student example

```
class student:
    college = “Naresh Tech"

    def __init__(self, no,name, sub1, sub2):
        self.no= no
        self.name = name
        self.sub1=sub1
        self.sub2=sub2

    def Compute_Avg(self):
        res = (self.sub1 + self.sub2)/2;
        print(f"College is  {student.college}")
        print(f"Number is {self.no}")
        print(f"Name is {self.name}")
        print(f"Average is {res}")
```

```
stud3 = student(100,'Rama', 70, 80)
stud3.Compute_Avg()
print()
stud4 = student(101,'Krishna', 50,60);
stud4.Compute_Avg()
```

Abstraction bank example

```
class Bank:

    def __init__(self, name):
        self.user_name = name
        self.balance = 0.0

    def show_balance(self):
        print(f"{self.user_name} has balanceRs.{self.balance} ")

    def withdraw_money(self, amount):
        if amount > self.balance:
            print("You don't have sufficient funds")
        else:
            self.balance -= amount
            print(f"{self.user_name} withdrawn
Rs.{self.balance}")
    def deposit_money(self, amount):
            self.balance += amount
            print(f"{self.user_name} deposited  Rs.{self.balance}
")
```

```
acc = Bank("Naresh")
acc.deposit_money(1000)
acc.show_balance()
acc.withdraw_money(500)
acc.show_balance()
```

Encapsulation

```
class Addition:

    def __init__(self, a, b):
        self.a = a
        self.b = b

    def add(self):
        return self.a + self.b

add_obj = Addition(3,4)
print(add_obj.add())
```

7

Encapsulation

Abstraction

Encapsulation

- **Getters**:- These are the methods used in Object-Oriented Programming (OOPS) which helps to access the private attributes from a **class**.
- **Setters**:- These are the methods used in OOPS feature which helps to set the value to private attributes in a **class**.
- **EncTest** has three methods.
- **__init__**:- It is used to initialize the attributes or properties of a **class**.
  - **__a**:- It is a private attribute.
- **get_a**:- It is used to get the values of private attribute **a**.
- **set_a**:- It is used to set the value of **a** using an **object** of a class.
- You are not able to access the private variables directly in **Python**. That's why you implemented the **getter** method.

Encapsulation

```
class EncTest:

    def __init__(self, a):
        self.__a = a

    def get_a(self):
        return self.__a

    def set_a(self, a):
        self.__a = a

e = EncTest(10)
print(e.get_a())

e.set_a(50)
print(e.get_a())
```

```
10
50
```

Default arguments

```
class student:

    def __init__(self, name= “Naresh Tech", test1= 80 , test2 = 70):
        self.name=name
        self.test1=test1
        self.test2=test2

    def Compute_Avg(self):
        res = (self.test1 + self.test2)/2;
        print(f"Name is {self.name}")
        print(f"Test1 is {self.test1}")
        print(f"Test2 is {self.test2}")
        print(f"Result is {res}")
```

```
stud1 = student()
stud1.Compute_Avg()

print()

stud2 = student(" NIT ", 40,50)
stud2.Compute_Avg()
```